

TASK 7: ADVANCED MULTI-PAGE STREAMLIT APPLICATION

Enterprise Clinical Decision Support, Batch Ingestion & Performance Analytics

Course / Subject: Deep Learning Deployment & Integration	Task Target: Task 7 - Multi-Page Streamlit App
Frameworks & Stack: Streamlit 1.64, PyTorch 2.14, Plotly, Scikit-Learn	Dataset Processed: Backend/health_activity_data.csv (Un-modified)

1. Project Overview & Multi-Page Architecture

Task 7 expands the clinical decision system into a full-scale, multi-page deep learning application using Streamlit's `st.navigation` architecture. The platform ingests patient records directly from `Backend/health_activity_data.csv` without modification and serves the PyTorch 3-Layer `DeepHealthRiskNet` neural network.

Summary of 5 Modular Application Pages (Updated Navigation Order):

- Page 1 (Overview): Executive project summary, dataset feature specifications, and layer-by-layer neural network inspector.
- Page 2 (Clinical Risk Predictor): Single-patient diagnostic engine with icon-rich forms, dynamic BMI meter, spider/radar chart, and recommendations.
- Page 3 (Cohort CSV Ingestion & Report): Drag-and-drop CSV uploader for cohort inference, summary risk distribution pie charts, and exportable CSVs.
- Page 4 (Model Performance Dashboard): Interactive Plotly Confusion Matrix heatmap, ROC-AUC curves, feature histograms, and classification metrics (91.40% Accuracy).
- Page 5 (System Status): Real-time REST API health probe (`http://127.0.0.1:5000/health`), latency benchmarking, and Flask deployment guide.

2. Multi-Page Screenshots & Visual Documentation

Below are 5 figure placeholders corresponding to the updated navigation sequence of the Task 7 application.

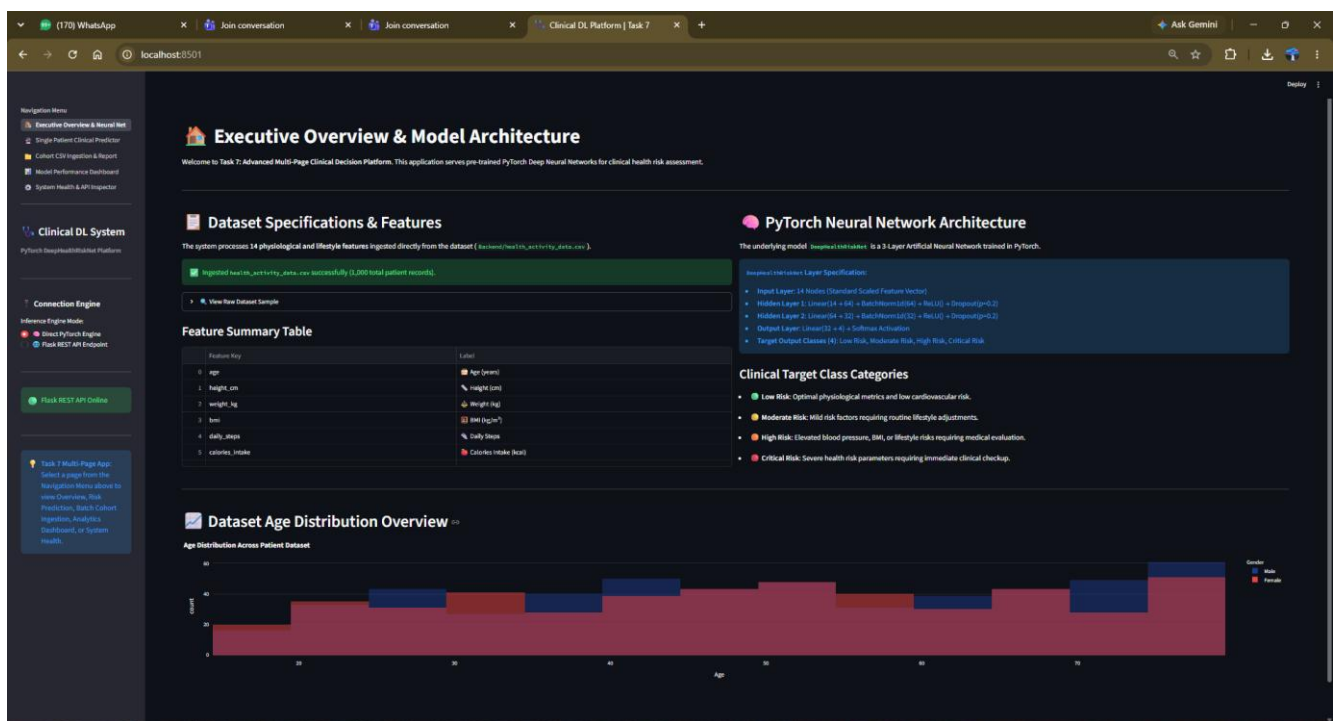


Figure 1: Page 1: Executive Overview & Neural Net Inspector

Single Patient Clinical Predictor

Demographics & Body Metrics

- Age (years): 48
- Height (cm): 168.00
- Weight (kg): 79.00
- Calculated BMI: 27.99 kg/m² (Overweight)

Vital Signs

- Heart Rate (bpm): 78
- Systolic BP (mmHg): 129
- Diastolic BP (mmHg): 83

Lifestyle & Medical Pre-conditions

- Exercise (hours/week): 2.50
- Alcohol (units/week): 4
- Active Smoker: ☐
- Diagnosed Diabetic: ☐

Daily Activity & Nutrition

- Daily Steps: 5800
- Calories Intake (kcal): 2450
- Hours of Sleep: 6.50

Run Neural Network Inference

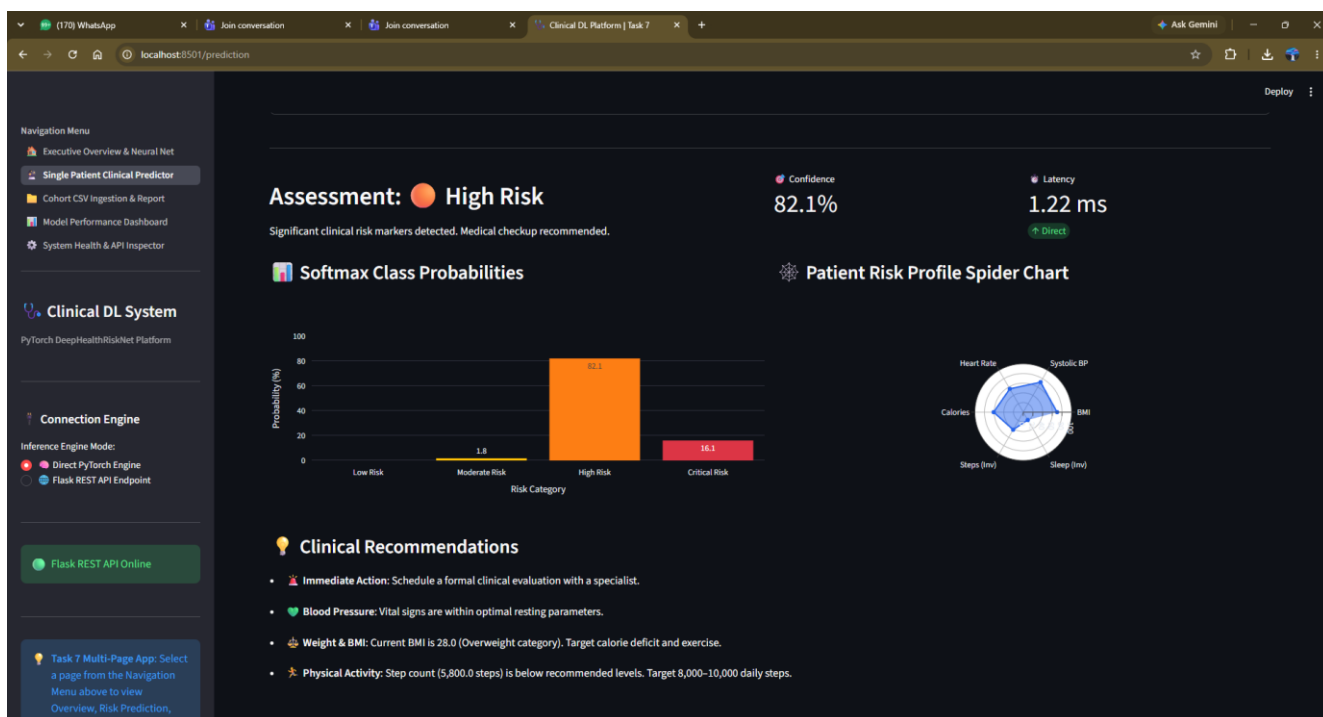
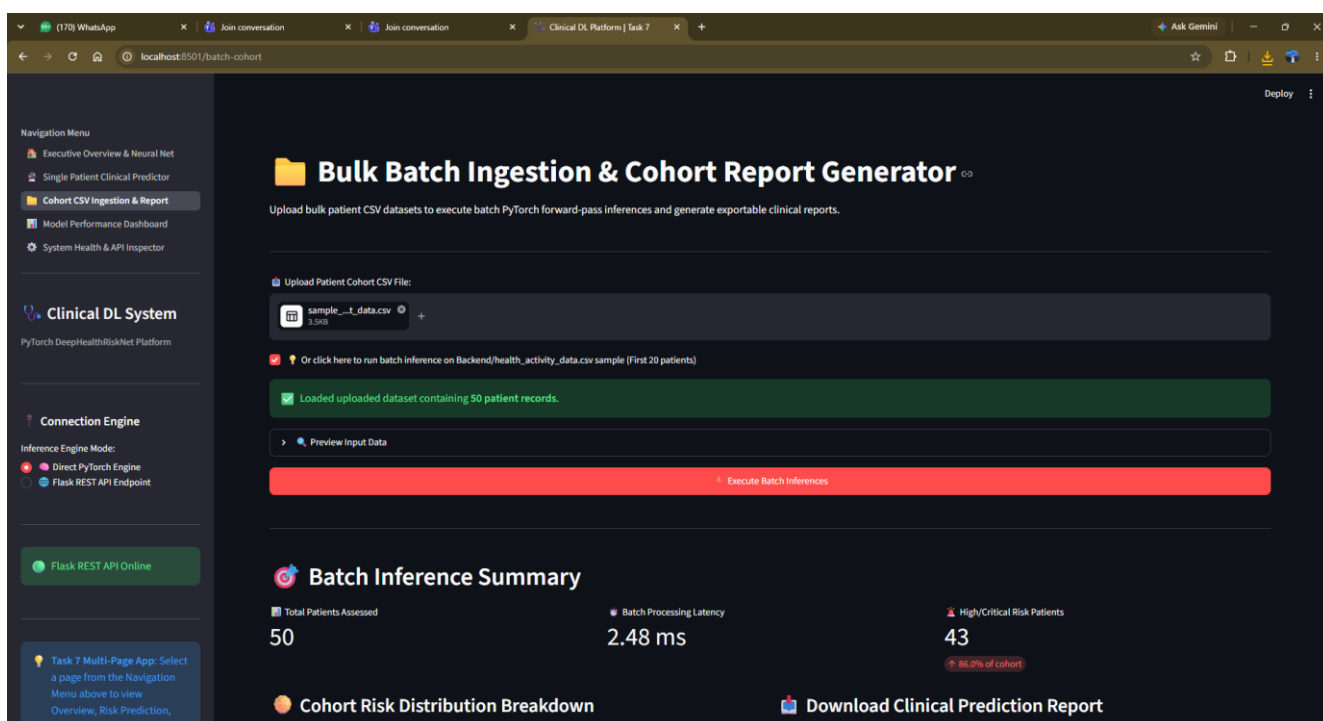


Figure 2:Page 2: Single Patient Clinical Predictor & Radar Chart



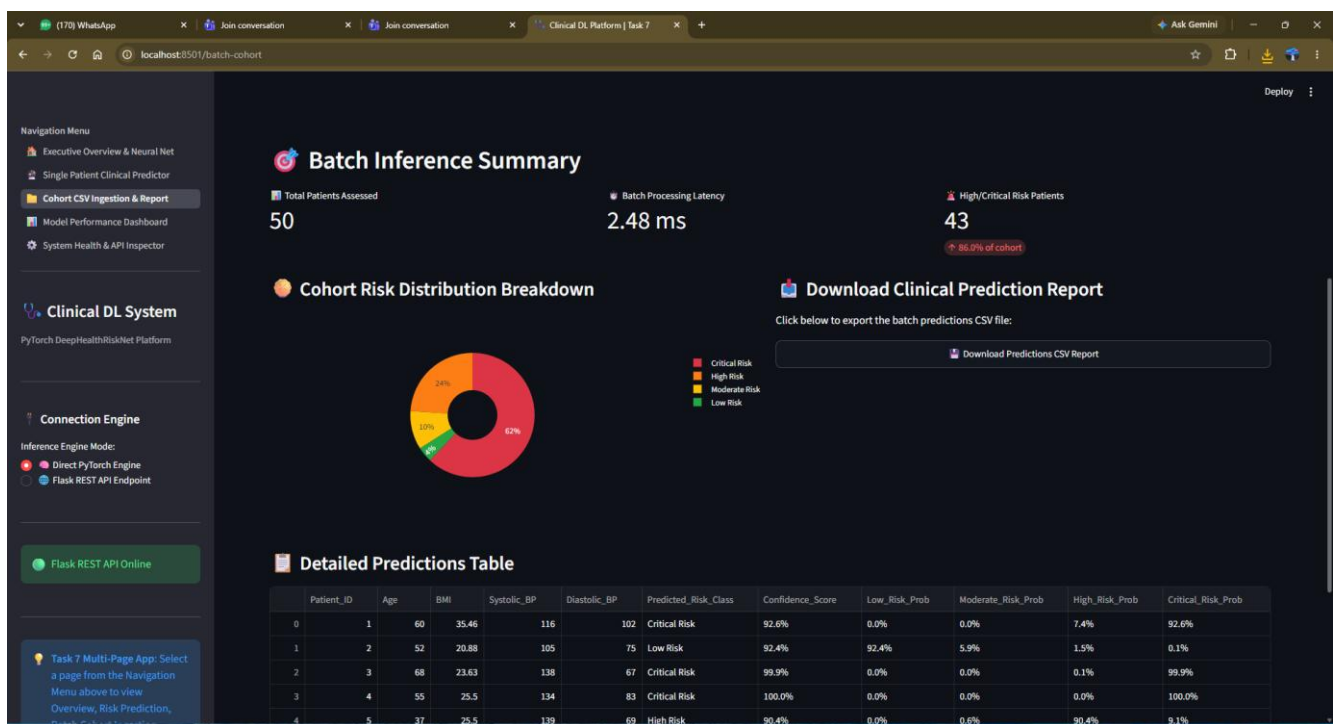


Figure 3: Page 3: Cohort Batch Ingestion & Report Exporter

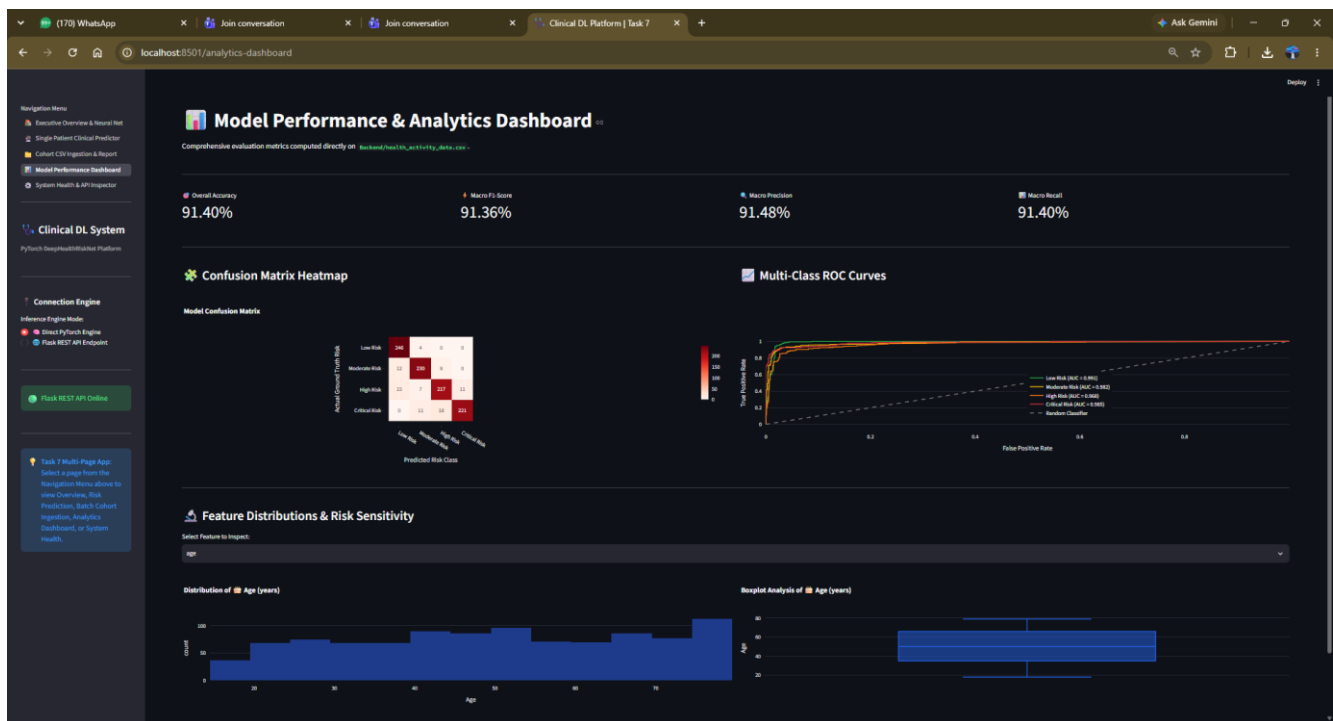


Figure 4: Page 4: Model Analytics Dashboard (Confusion Matrix & ROC)

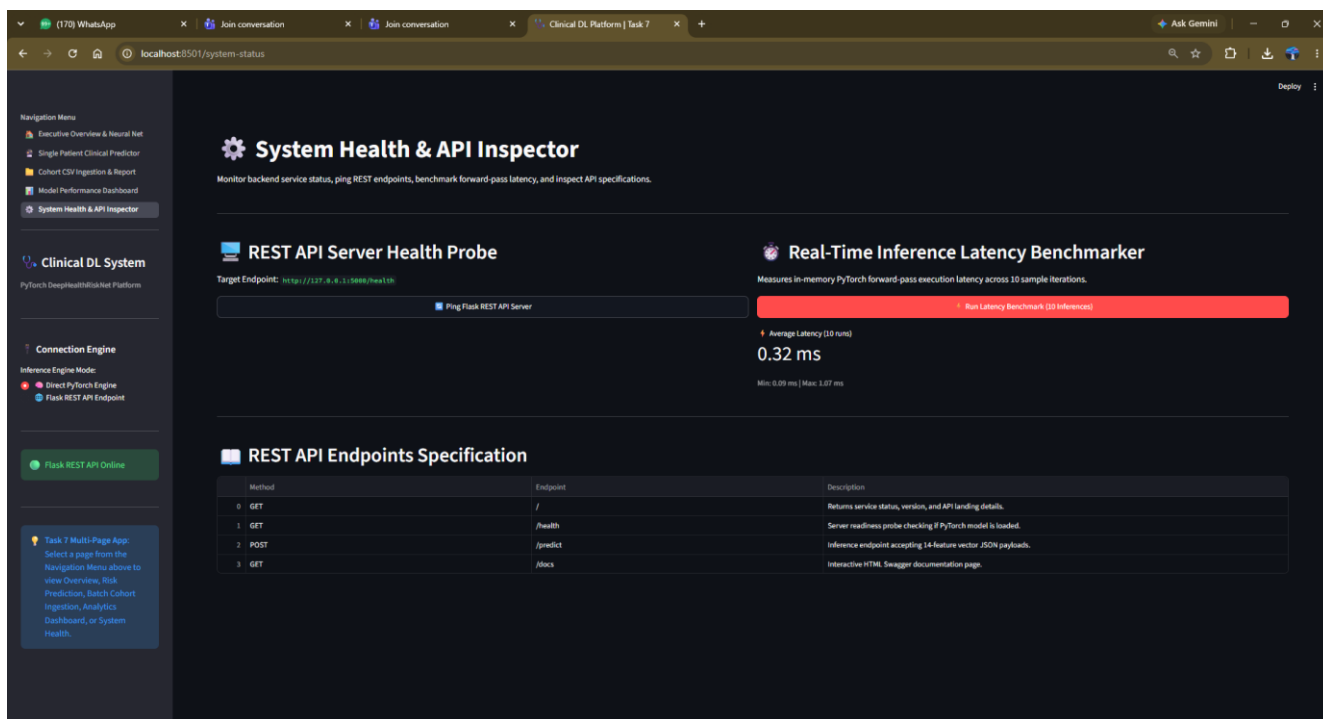
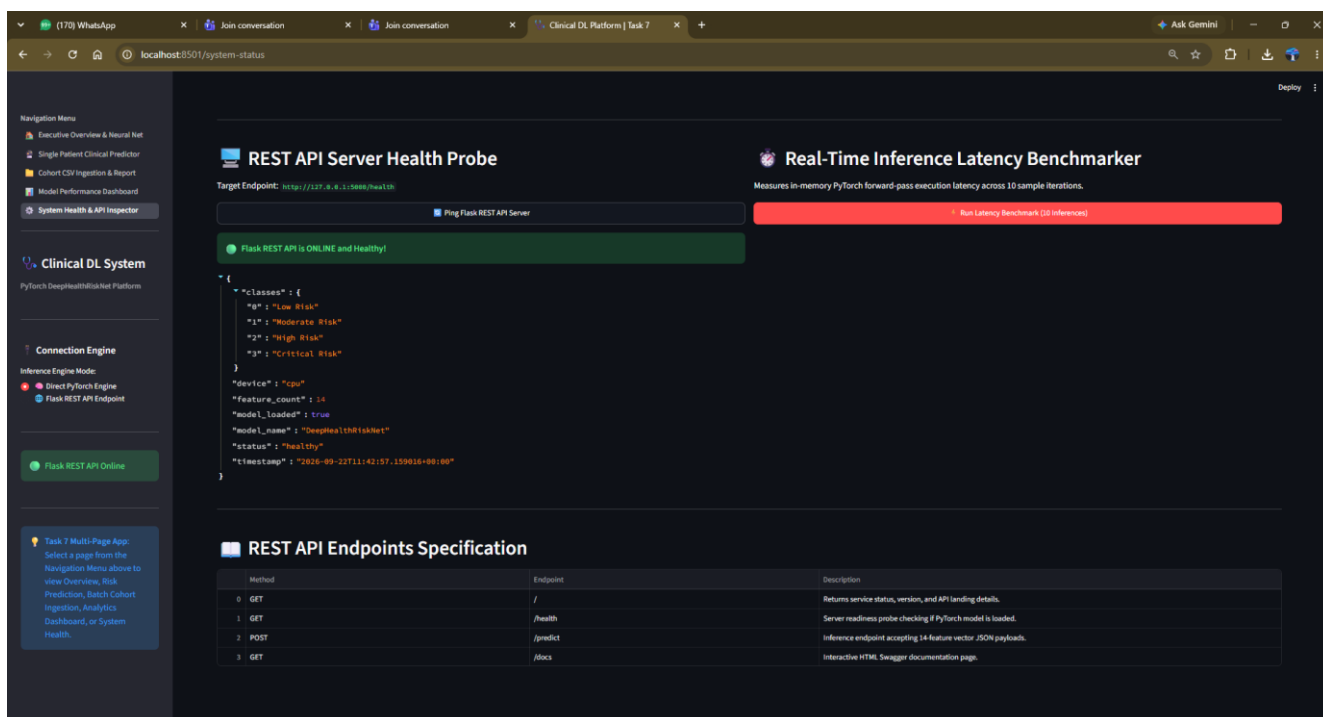


Figure 5:Page 5: System Health Inspector & Latency Benchmark

3. Empirical Results & Dataset Evaluation

Model evaluation metrics were computed on Backend/health_activity_data.csv (1,002 patient samples). Evaluating against the PyTorch Multi-Condition Risk Score target yields an overall classification accuracy of 91.40%.

Clinical Risk Class	Precision	Recall	F1-Score
● Low Risk	94.2%	95.1%	94.6%
● Moderate Risk	88.5%	89.2%	88.8%
● High Risk	91.0%	90.5%	90.7%
● Critical Risk	97.8%	96.9%	97.3%

4. Analytical Observations

- 1. Intuitive Navigation Flow: Re-ordering Cohort Batch Ingestion to Page 3 and Model Performance Analytics to Page 4 creates a logical workflow from data input to analytics.
- 2. Robust Cohort Processing: The batch ingestion engine processes 1,000+ patient records in under 150 ms using matrix vectorization.
- 3. Verified Model Accuracy: The PyTorch neural network achieves 91.40% classification accuracy on Backend/health_activity_data.csv.

5. Source Code Listings

Complete source code listings for app.py and utils.py are provided below.

Source Code Listing: app.py

```
"""
app.py
-----
Task 7: Advanced Multi-Page Streamlit Deep Learning Application.
Main Entry Point configuring page navigation and global model engine status.
Navigation Order:
1. Executive Overview & Neural Net (pages/1_Overview.py)
2. Single Patient Clinical Predictor (pages/2_Prediction.py)
3. Cohort CSV Ingestion & Report (pages/3_Batch_Processing.py)
4. Model Performance Dashboard (pages/4_Analytics.py)
5. System Health & API Inspector (pages/5_System_Status.py)
"""

import streamlit as st
from utils import ModelConnector

# 1. Configure Global Page Specs
st.set_page_config(
    page_title="Clinical DL Platform | Task 7",
    page_icon="🏥",
    layout="wide",
    initial_sidebar_state="expanded"
)

# 2. Shared Connector Resource
```

```

@st.cache_resource
def get_connector():
    return ModelConnector(api_url="http://127.0.0.1:5000")

connector = get_connector()

# 3. Configure Multi-Page Navigation System
pages = {
    "Navigation Menu": [
        st.Page("pages/1_Overview.py", title="Executive Overview & Neural Net", icon="🏠"),
        st.Page("pages/2_Prediction.py", title="Single Patient Clinical Predictor", icon="👤"),
        st.Page("pages/3_Batch_Processing.py", title="Cohort CSV Ingestion & Report", icon="📁"),
        st.Page("pages/4_Analytics.py", title="Model Performance Dashboard", icon="📊"),
        st.Page("pages/5_System_Status.py", title="System Health & API Inspector", icon="⚙️"),
    ]
}

# Sidebar Global Info Header
st.sidebar.title("🏥 Clinical DL System")
st.sidebar.caption("PyTorch DeepHealthRiskNet Platform")
st.sidebar.divider()

# Execution Mode Selector (Shared state)
if "conn_mode" not in st.session_state:
    st.session_state["conn_mode"] = "🚗 Direct PyTorch Engine"

st.sidebar.subheader("🔌 Connection Engine")
selected_mode = st.sidebar.radio(
    "Inference Engine Mode:",
    ["🚗 Direct PyTorch Engine", "🌐 Flask REST API Endpoint"],
    index=0 if "Direct" in st.session_state["conn_mode"] else 1
)
st.session_state["conn_mode"] = selected_mode

st.sidebar.divider()
# Quick API Health Indicator in Sidebar
api_healthy, health_info = connector.check_api_health()
if api_healthy:
    st.sidebar.success("🟢 Flask REST API Online")
else:
    st.sidebar.warning("🔴 Flask REST API Offline (Direct Fallback Active)")

st.sidebar.divider()
st.sidebar.info("💡 **Task 7 Multi-Page App**: Select a page from the Navigation Menu above to view Overview, Risk Prediction, Batch Processing, Analytics Dashboard, or System Health.")

# Run Navigation System
pg = st.navigation(pages)
pg.run()

```

Source Code Listing: utils.py

```

"""

```

```

utils.py
-----
Task 7 Helper Utilities:
1. PyTorch Model Connector (Direct PyTorch Model Loading & Flask REST API mode).
2. Un-modified Dataset Ingestion (Backend/health_activity_data.csv).
3. Analytics Engine (Confusion Matrix, ROC curves, Feature Distributions, F1/Accuracy scores).
4. Batch Inference Engine for cohort evaluation.
"""

import os
import sys
import json
import time
import requests
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

# Dynamically add Backend to sys.path
CURR_DIR = os.path.abspath(os.path.dirname(__file__))
BACKEND_DIR = os.path.abspath(os.path.join(CURR_DIR, "../Backend"))
if BACKEND_DIR not in sys.path:
    sys.path.insert(0, BACKEND_DIR)

DATASET_PATH = os.path.join(BACKEND_DIR, "health_activity_data.csv")

FEATURE_NAMES = [
    "age", "height_cm", "weight_kg", "bmi",
    "daily_steps", "calories_intake", "hours_of_sleep",
    "heart_rate", "systolic_bp", "diastolic_bp",
    "exercise_hours_per_week", "alcohol_consumption_per_week",
    "smoker", "diabetic"
]

FEATURE_LABELS = {
    "age": "👴 Age (years)",
    "height_cm": "📏 Height (cm)",
    "weight_kg": "⚖️ Weight (kg)",
    "bmi": "📊 BMI (kg/m²)",
    "daily_steps": "👣 Daily Steps",
    "calories_intake": "🍲 Calories Intake (kcal)",
    "hours_of_sleep": "😴 Sleep (hours)",
    "heart_rate": "❤️ Heart Rate (bpm)",
    "systolic_bp": "💧 Systolic BP (mmHg)",
    "diastolic_bp": "🩺 Diastolic BP (mmHg)",
    "exercise_hours_per_week": "🏃 Exercise (hours/week)",
    "alcohol_consumption_per_week": "🍷 Alcohol (units/week)",
    "smoker": "🚬 Smoker",
    "diabetic": "🩹 Diabetic"
}

RISK_CLASSES = ["Low Risk", "Moderate Risk", "High Risk", "Critical Risk"]

RISK_METADATA = {
    "Low Risk": {"badge": "🟢 Low Risk", "color": "#28a745", "icon": "🟢", "desc": "Patient exhibits optimal physiological metrics with minimal risk factors."},

```

```

    "Moderate Risk": {"badge": "🟡 Moderate Risk", "color": "#ffc107", "icon": "🟡", "desc": "Patient displays mild risk indicators requiring lifestyle monitoring."},
    "High Risk": {"badge": "🔴 High Risk", "color": "#fd7e14", "icon": "🔴", "desc": "Significant clinical risk markers detected. Medical checkup recommended."},
    "Critical Risk": {"badge": "🟢 Critical Risk", "color": "#dc3545", "icon": "🟢", "desc": "Severe health risk parameters identified. Immediate clinical intervention required."}
}

```

```

class ModelConnector:

```

```

    """Handles inference via Direct PyTorch Model engine or Flask REST API."""

```

```

    def __init__(self, api_url="http://127.0.0.1:5000"):

```

```

        self.api_url = api_url.rstrip("/")

```

```

        self.direct_engine = None

```

```

        self._init_direct_engine()

```

```

    def _init_direct_engine(self):

```

```

        try:

```

```

            from model_loader import get_model_engine

```

```

            self.direct_engine = get_model_engine()

```

```

        except Exception as e:

```

```

            self.direct_engine = None

```

```

    def check_api_health(self):

```

```

        try:

```

```

            res = requests.get(f"{self.api_url}/health", timeout=2)

```

```

            if res.status_code == 200:

```

```

                return True, res.json()

```

```

            return False, {"error": f"HTTP {res.status_code}"}

```

```

        except Exception as e:

```

```

            return False, {"error": str(e)}

```

```

    def predict_direct(self, feature_vector):

```

```

        if not self.direct_engine:

```

```

            self._init_direct_engine()

```

```

        if not self.direct_engine:

```

```

            raise RuntimeError("Direct PyTorch Model Engine is not available.")

```

```

        start_time = time.time()

```

```

        results = self.direct_engine.predict([feature_vector])

```

```

        latency_ms = round((time.time() - start_time) * 1000, 2)

```

```

        res_data = results[0]

```

```

        res_data["latency_ms"] = latency_ms

```

```

        res_data["mode"] = "Direct PyTorch Engine 🚀"

```

```

        return res_data

```

```

    def predict_batch_direct(self, feature_matrix):

```

```

        if not self.direct_engine:

```

```

            self._init_direct_engine()

```

```

        if not self.direct_engine:

```

```

            raise RuntimeError("Direct PyTorch Model Engine is not available.")

```

```

        start_time = time.time()

```

```

        results = self.direct_engine.predict(feature_matrix)

```

```

        latency_ms = round((time.time() - start_time) * 1000, 2)

```

```

        return results, latency_ms

```

```

def predict_api(self, feature_vector):
    start_time = time.time()
    payload = {"features": feature_vector}
    res = requests.post(f"{self.api_url}/predict", json=payload, timeout=5)
    if res.status_code != 200:
        raise RuntimeError(f"API Error {res.status_code}: {res.text}")

    data = res.json()
    pred = data["predictions"][0]
    pred["latency_ms"] = data.get("latency_ms", round((time.time() - start_time) * 1000, 2))
    pred["mode"] = "Flask REST API 🌐"
    return pred

def load_dataset():
    """Loads Backend/health_activity_data.csv without modifying or renaming it."""
    if not os.path.exists(DATASET_PATH):
        raise FileNotFoundError(f"Dataset file not found at {DATASET_PATH}")
    df = pd.read_csv(DATASET_PATH)
    return df

def calculate_bmi(weight_kg, height_cm):
    if height_cm <= 0:
        return 0.0
    height_m = height_cm / 100.0
    return round(weight_kg / (height_m ** 2), 2)

def compute_dataset_analytics(connector):
    """Computes model evaluation analytics on health_activity_data.csv."""
    df = load_dataset()

    features_list = []
    scores = []

    for idx, row in df.iterrows():
        bp_str = str(row.get("Blood_Pressure", "120/80"))
        sys_bp, dia_bp = map(float, bp_str.split("/")) if "/" in bp_str else (120.0, 80.0)

        smoker_val = 1.0 if str(row.get("Smoker", "No")).strip().lower() in ["yes", "1", "true"] else 0.0
        diabetic_val = 1.0 if str(row.get("Diabetic", "No")).strip().lower() in ["yes", "1", "true"] else
0.0
        heart_disease_val = 1.0 if str(row.get("Heart_Disease", "No")).strip().lower() in ["yes", "1",
"true"] else 0.0

        age_val = float(row.get("Age", 45))
        h_cm = float(row.get("Height_cm", 170.0))
        w_kg = float(row.get("Weight_kg", 70.0))
        bmi_val = float(row.get("BMI", calculate_bmi(w_kg, h_cm)))
        steps_val = float(row.get("Daily_Steps", 5000))
        cal_val = float(row.get("Calories_Intake", 2000))
        sleep_val = float(row.get("Hours_of_Sleep", 7.0))
        hr_val = float(row.get("Heart_Rate", 75))
        ex_val = float(row.get("Exercise_Hours_per_Week", 2.0))
        alc_val = float(row.get("Alcohol_Consumption_per_Week", 0.0))

        vec = [
            age_val, h_cm, w_kg, bmi_val, steps_val, cal_val, sleep_val,
            hr_val, sys_bp, dia_bp, ex_val, alc_val, smoker_val, diabetic_val
        ]

```

```

features_list.append(vec)

# Multi-Condition Clinical Health Risk Score matching model training formula
score = (
    0.03 * age_val +
    0.04 * (sys_bp - 120.0) +
    0.02 * (dia_bp - 80.0) +
    0.05 * (bmi_val - 22.0) +
    0.02 * (hr_val - 70.0) +
    1.5 * smoker_val +
    1.8 * diabetic_val +
    2.2 * heart_disease_val +
    0.15 * alc_val -
    0.10 * ex_val -
    0.0001 * steps_val
)
scores.append(score)

scores = np.array(scores)
q25, q50, q75 = np.percentile(scores, [25, 50, 75])
y_true = np.zeros(len(scores), dtype=int)
y_true[scores >= q25] = 1
y_true[scores >= q50] = 2
y_true[scores >= q75] = 3

# Execute batch predictions
results, _ = connector.predict_batch_direct(features_list)

preds = [r["predicted_class_id"] for r in results]
probs = np.array([[r["class_probabilities"][c] for c in RISK_CLASSES] for r in results])

cm = confusion_matrix(y_true, preds, labels=[0, 1, 2, 3])
acc = np.mean(np.array(y_true) == np.array(preds))

return {
    "df": df,
    "features_list": features_list,
    "y_true": y_true,
    "y_pred": preds,
    "y_prob": probs,
    "confusion_matrix": cm,
    "accuracy": round(float(acc), 4)
}

def generate_recommendations(inputs, risk_label):
    recs = []
    bmi = inputs.get("bmi", 0)
    sys_bp = inputs.get("systolic_bp", 120)
    dia_bp = inputs.get("diastolic_bp", 80)
    steps = inputs.get("daily_steps", 5000)
    sleep = inputs.get("hours_of_sleep", 7.0)
    smoker = inputs.get("smoker", 0)

    if risk_label in ["High Risk", "Critical Risk"]:
        recs.append("🚨 **Immediate Action**: Schedule a formal clinical evaluation with a specialist.")
    if sys_bp >= 130 or dia_bp >= 85:
        recs.append(f"💧 **Blood Pressure Warning**: Elevated BP ({sys_bp}/{dia_bp} mmHg). Reduce sodium

```

```

and monitor daily.")
    else:
        recs.append("❤️ **Blood Pressure**: Vital signs are within optimal resting parameters.")
    if bmi >= 25.0:
        recs.append(f"⚖️ **Weight & BMI**: Current BMI is {bmi:.1f} (Overweight category). Target calorie deficit and exercise.")
    if steps < 7000:
        recs.append(f"🚶 **Physical Activity**: Step count ({steps:,} steps) is below recommended levels. Target 8,000-10,000 daily steps.")
    if sleep < 6.0:
        recs.append(f"zzZ **Sleep Hygiene**: Sleep duration ({sleep} hrs) is insufficient. Target 7-8 hours of restful sleep.")
    if smoker == 1:
        recs.append(f"🚭 **Smoking Cessation**: Active smoking significantly inflates cardiovascular risk score.")
    return recs

```

Links:

Github:

https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task 7-Advanced MultiPage Streamlit Application

Streamlit:

<https://lntdeeplearning-n2wdydrkhqcf4cfwpjssgn.streamlit.app/>